MODUL PRAKTIKUM

PRAKTIKUM DASAR PEMROGRAMAN

Modul 7

Fungsi, Pemrograman Modular, dan Rekursi Dasar

Disusun oleh:

Dr. Desta Sandya Prasvita, S.Komp., M.Kom.

PROGRAM STUDI S1 INFORMATIKA

UPN VETERAN JAKARTA

2026

Modul ini melanjutkan materi seleksi, perulangan, dan debugging pada modul sebelumnya. Fokus praktikum adalah memecah program menjadi beberapa fungsi yang mempunyai tanggung jawab jelas sehingga program lebih terstruktur, mudah dibaca, diuji, diperbaiki, dan digunakan kembali.

Modul ini terdiri atas tiga bagian besar. Bagian pertama membahas fungsi (deklarasi, definisi, parameter, dan nilai kembalian). Bagian kedua membahas pemrograman modular (dekomposisi masalah dan pembagian tanggung jawab antar fungsi). Bagian ketiga memperkenalkan rekursi, yaitu fungsi yang memanggil dirinya sendiri, sebagai bentuk lanjutan dari konsep fungsi yang telah dipelajari.

Rekursi akan digunakan kembali pada Modul 13 (Binary Search Rekursif) sebagai pengayaan, sehingga pemahaman yang kuat terhadap base case dan tracing pemanggilan pada modul ini akan sangat membantu.

I. Pendahuluan Modul

1. Identitas Modul

Komponen	Keterangan
Mata Kuliah	Praktikum Dasar Pemrograman
Program Studi	S1 Informatika
Semester	1
Modul	7
Judul	Fungsi, Pemrograman Modular, dan Rekursi Dasar
Bahasa Pemrograman	C
Alokasi Waktu	1 × 170 menit
Metode	Demonstrasi, live coding, latihan terbimbing, tracing, eksperimen, debugging, dan latihan mandiri
Prasyarat	Modul 1–6

2. Indikator Keberhasilan

Mahasiswa dinyatakan berhasil apabila mampu:

- mengidentifikasi bagian program yang dapat dijadikan fungsi;
- memberi nama fungsi yang menggambarkan tugasnya;
- menulis prototype sesuai definisi fungsi;
- memanggil fungsi dengan jumlah dan tipe argumen yang tepat;
- membuat fungsi bertipe void;
- membuat fungsi bertipe int, double, atau char;
- menggunakan parameter untuk menerima data;
- mengembalikan hasil menggunakan return;
- menjelaskan bahwa perubahan parameter biasa tidak mengubah variabel pemanggil;
- membatasi penggunaan variabel global;
- menggunakan variabel lokal secara tepat;
- mengembangkan program dengan minimal lima fungsi;
- menentukan base case pada fungsi rekursif;
- menentukan recursive case pada fungsi rekursif;
- melakukan tracing terhadap pemanggilan rekursif secara bertahap;
- memprediksi nilai kembalian dari suatu pemanggilan rekursif;
- mendeteksi potensi infinite recursion pada suatu fungsi;
- membandingkan solusi iteratif dan solusi rekursif untuk masalah yang sama;
- memilih kapan rekursi layak digunakan dan kapan iterasi lebih sesuai;

- menguji fungsi menggunakan beberapa nilai masukan;
- menjelaskan alur program dari main() ke fungsi lain, termasuk ke dirinya sendiri pada kasus rekursi.

II. Persiapan Praktikum

1. Persiapan Folder Praktikum

Buat struktur folder berikut:

```
Praktikum_Dasar_Pemrograman/
└── Modul_07/
    ├── latihan_01.c
    ├── latihan_02.c
    ├── latihan_03.c
    ├── tracing.c
    ├── eksperimen.c
    ├── debugging_01.c
    ├── debugging_02.c
    └── tugas_modul_07.c
```

Ketentuan nama berkas:

- menggunakan huruf kecil;
- tidak menggunakan spasi;
- menggunakan garis bawah _;
- menggunakan ekstensi .c.

III. Konsep Dasar

Bagian ini terbagi menjadi tiga kelompok besar: Bagian A membahas Fungsi, Bagian B membahas Pemrograman Modular, dan Bagian C membahas Rekursi.

Bagian A — Fungsi

1. Struktur Fungsi dalam Bahasa C

Sebuah fungsi dapat memiliki: tipe nilai kembalian, nama fungsi, daftar parameter, badan fungsi, dan pernyataan return (apabila fungsi mengembalikan nilai).

Bentuk umum:

```
tipe_kembalian nama_fungsi(daftar_parameter)
{
    pernyataan;

    return nilai;
}
```

Contoh:

```
double hitung_luas(double panjang, double lebar)
{
```

```
    double luas;

    luas = panjang * lebar;

    return luas;
}
```

Bagian-bagiannya:

Bagian	Contoh
Tipe nilai kembalian	double
Nama fungsi	hitung_luas
Parameter	double panjang, double lebar
Variabel lokal	double luas
Nilai kembalian	return luas;

2. Tiga Tahap Penggunaan Fungsi

Penggunaan fungsi umumnya melibatkan: deklarasi atau prototype, definisi, dan pemanggilan.

a. Deklarasi atau Function Prototype

Prototype memberi tahu compiler mengenai nama fungsi, tipe nilai kembalian, jumlah parameter, dan tipe setiap parameter.

Contoh:

```
double hitung_luas(double panjang, double lebar);
```

Nama parameter pada prototype dapat dihilangkan:

```
double hitung_luas(double, double);
```

Namun, nama parameter disarankan karena membuat prototype lebih mudah dipahami.

Prototype diakhiri dengan titik koma.

b. Definisi Fungsi

Definisi berisi proses yang dijalankan oleh fungsi.

```
double hitung_luas(double panjang, double lebar)
{
    return panjang * lebar;
}
```

Definisi tidak diakhiri dengan titik koma setelah kurung kurawal penutup.

c. Pemanggilan Fungsi

Fungsi dipanggil menggunakan nama fungsi dan argumen.

```
luas = hitung_luas(panjang, lebar);
```

Pada pemanggilan tersebut: fungsi hitung_luas() dijalankan, nilai panjang dan lebar dikirim, fungsi menghitung hasil, hasil dikembalikan, dan hasil disimpan dalam variabel luas.

3. Posisi Prototype dan Definisi

Susunan yang disarankan:

```
#include <stdio.h>

/* Function prototype */
double hitung_luas(double panjang, double lebar);

int main(void)
{
    double hasil;

    hasil = hitung_luas(10.0, 5.0);

    printf("Luas: %.2f\n", hasil);

    return 0;
}

/* Function definition */
double hitung_luas(double panjang, double lebar)
{
    return panjang * lebar;
}
```

Prototype ditempatkan sebelum main(), sedangkan definisi ditempatkan setelah main().

Tanpa prototype, compiler belum mengetahui fungsi yang dipanggil apabila definisinya berada setelah main().

Untuk program dengan banyak fungsi, penggunaan prototype membuat susunan program lebih teratur.

4. Fungsi void

Fungsi bertipe void tidak mengembalikan suatu nilai kepada pemanggil.

Contoh:

```
void tampilkan_judul(void)
{
    printf("================================\n");
    printf("     PRAKTIKUM PEMROGRAMAN      \n");
    printf("================================\n");
}
```

Pemanggilan:

```
tampilkan_judul();
```

Fungsi void cocok untuk:

- menampilkan informasi;
- mencetak menu;
- mencetak garis pemisah;
- menjalankan proses yang tidak perlu menghasilkan satu nilai.

5. Fungsi Tanpa Parameter dan Fungsi dengan Parameter

Fungsi tanpa parameter ditulis menggunakan void pada daftar parameter.

```
void tampilkan_menu(void)
{
    printf("1. Tambah\n");
    printf("2. Kurang\n");
    printf("3. Keluar\n");
}
```

Parameter digunakan untuk menerima data dari pemanggil.

```
void tampilkan_bilangan(int bilangan)
{
    printf("Bilangan: %d\n", bilangan);
}
```

Pemanggilan:

```
tampilkan_bilangan(10);
```

Urutan argumen harus sesuai dengan urutan parameter.

6. Parameter Formal dan Argumen Aktual

Perhatikan fungsi:

```
double hitung_luas(double panjang, double lebar)
{
    return panjang * lebar;
}
```

Parameter double panjang dan double lebar disebut parameter formal.

Pemanggilan:

```
luas = hitung_luas(p, l);
```

Variabel p dan l disebut argumen aktual atau argumen.

Pemetaan:

Argumen aktual	Parameter formal
p	panjang
l	lebar

Urutan sangat penting.

```
hasil = hitung_pembagian(angka_1, angka_2);
```

berbeda dengan:

```
hasil = hitung_pembagian(angka_2, angka_1);
```

7. Fungsi dengan Nilai Kembalian

Fungsi yang menghasilkan nilai harus menggunakan tipe kembalian yang sesuai.

Mengembalikan int

```
int hitung_kuadrat(int bilangan)
{
    return bilangan * bilangan;
}
```

Mengembalikan double

```
double hitung_rata_rata(double total, int jumlah_data)
{
    return total / jumlah_data;
}
```

Mengembalikan char

```
char tentukan_huruf_mutu(double nilai)
{
    if (nilai >= 85.0)
    {
        return 'A';
    }
    else if (nilai >= 75.0)
    {
        return 'B';
    }

    return 'E';
}
```

8. Pernyataan return dan Pass by Value

return digunakan untuk mengembalikan nilai dan menghentikan fungsi. Kode setelah return tidak dijalankan.

Early Return

```
double hitung_pembagian(double pembilang, double penyebut)
{
    if (penyebut == 0.0)
    {
        printf("Penyebut tidak boleh nol.\n");
        return 0.0;
    }

    return pembilang / penyebut;
}
```

Pass by Value

Bahasa C mengirimkan argumen biasa ke fungsi menggunakan pass by value. Artinya, fungsi menerima salinan nilai.

```
void tambah_satu(int nilai)
{
    nilai++;
    printf("Nilai di fungsi: %d\n", nilai);
}
```

Perubahan terhadap nilai tidak mengubah variabel pemanggil.

Untuk mengubah variabel milik pemanggil secara langsung diperlukan alamat memori atau pointer. Materi tersebut dibahas pada Modul 11.

9. Variabel Lokal dan Variabel Global

Variabel lokal adalah variabel yang dideklarasikan di dalam suatu fungsi atau blok.

```
double hitung_luas(double panjang, double lebar)
{
    double luas;

    luas = panjang * lebar;

    return luas;
}
```

Variabel luas hanya dapat digunakan di dalam fungsi hitung_luas(). Dua fungsi berbeda dapat memiliki variabel lokal dengan nama yang sama tanpa saling memengaruhi.

Variabel global dideklarasikan di luar seluruh fungsi dan dapat diakses oleh beberapa fungsi dalam file yang sama.

```
int jumlah_pemanggilan = 0;

void tampilkan_pesan(void)
{
    jumlah_pemanggilan++;
    printf("Fungsi dipanggil.\n");
}
```

Namun, penggunaan variabel global harus dibatasi karena fungsi menjadi bergantung pada keadaan di luar dirinya, aliran data lebih sulit ditelusuri, dan debugging menjadi lebih sulit.

Lebih disarankan menggunakan parameter dan nilai kembalian.

10. Fungsi yang Memanggil Fungsi Lain

Satu fungsi dapat memanggil fungsi lain.

```
double hitung_subtotal(double harga, int jumlah)
{
    return harga * jumlah;
```

```
}

double hitung_total(
    double harga,
    int jumlah,
    double persentase_biaya
)
{
    double subtotal = hitung_subtotal(harga, jumlah);

    return subtotal + (subtotal * persentase_biaya);
}
```

Alur:

```
main()
  └── hitung_total()
        └── hitung_subtotal()
```

Prototype seluruh fungsi yang digunakan harus tersedia sebelum pemanggilan.

Alur pemanggilan seperti ini adalah jembatan menuju konsep rekursi pada Bagian C: bagaimana jika suatu fungsi memanggil dirinya sendiri, bukan fungsi lain?

Bagian B — Pemrograman Modular

11. Konsep Pemrograman Modular

Pemrograman modular adalah pendekatan mengembangkan program dengan membaginya menjadi bagian-bagian kecil yang disebut modul atau fungsi.

Program yang belum modular:

```
#include <stdio.h>

int main(void)
{
    double panjang;
    double lebar;
    double luas;
    double keliling;

    printf("Masukkan panjang: ");
    scanf("%lf", &panjang);

    printf("Masukkan lebar: ");
    scanf("%lf", &lebar);

    luas = panjang * lebar;
    keliling = 2.0 * (panjang + lebar);

    printf("Luas     : %.2f\n", luas);
```

```
    printf("Keliling : %.2f\n", keliling);

    return 0;
}
```

Program tersebut masih mudah dipahami karena ukurannya kecil. Namun, apabila jumlah proses bertambah, fungsi main() dapat menjadi terlalu panjang.

Program modular memecah proses tersebut menjadi hitung_luas(), hitung_keliling(), dan tampilkan_hasil() — setiap fungsi memiliki tugas yang lebih spesifik.

12. Manfaat Pemrograman Modular

- program lebih mudah dibaca — nama fungsi dapat menjelaskan tujuan proses;
- mengurangi pengulangan kode — proses yang berulang cukup ditulis dalam satu fungsi;
- memudahkan pengujian — setiap fungsi dapat diuji secara terpisah;
- memudahkan debugging — kesalahan dapat dicari pada fungsi yang menangani proses tertentu;
- memudahkan perubahan — perubahan rumus cukup dilakukan pada fungsi terkait;
- mendukung kerja tim — anggota tim dapat mengembangkan fungsi berbeda dengan kesepakatan prototype yang sama;
- meningkatkan penggunaan kembali kode — fungsi umum dapat digunakan dalam program lain.

13. Dekomposisi Masalah

Dekomposisi adalah proses membagi masalah besar menjadi beberapa submasalah.

Contoh program pengolahan nilai dapat dibagi menjadi:

```
Program Pengolahan Nilai
├── membaca nilai
├── memvalidasi nilai
├── menghitung nilai akhir
├── menentukan huruf mutu
├── menentukan status kelulusan
└── menampilkan hasil
```

Dalam bahasa C, submasalah tersebut dapat diwujudkan sebagai fungsi:

```
double baca_nilai(void);
double hitung_nilai_akhir(
    double tugas,
    double uts,
    double uas
);
char tentukan_huruf_mutu(double nilai_akhir);
int tentukan_kelulusan(double nilai_akhir);
void tampilkan_hasil(
    double nilai_akhir,
    char huruf_mutu,
    int status_lulus
```

```
);
```

Fungsi yang baik sebaiknya memiliki satu tanggung jawab utama (single responsibility).

Kurang baik:

```
void proses_semua_data(void);
```

Lebih baik:

```
double hitung_nilai_akhir(...);
char tentukan_huruf_mutu(...);
void tampilkan_hasil(...);
```

14. Fungsi Validasi, Fungsi Perhitungan, dan Fungsi Output

Dalam pemrograman modular, fungsi sering dikelompokkan berdasarkan perannya.

a. Fungsi validasi/input

Bertugas menerima dan memastikan data valid sebelum digunakan.

```
int baca_bilangan_dalam_rentang(
    int nilai_minimum,
    int nilai_maksimum
)
{
    int nilai;

    do
    {
        scanf("%d", &nilai);

        if ((nilai < nilai_minimum) ||
            (nilai > nilai_maksimum))
        {
            printf(
                "Input harus berada pada rentang %d-%d.\n",
                nilai_minimum,
                nilai_maksimum
            );
        }
    }
    while ((nilai < nilai_minimum) ||
           (nilai > nilai_maksimum));

    return nilai;
}
```

b. Fungsi perhitungan

Bertugas mengolah data dan mengembalikan hasil, tanpa menampilkan apa pun.

```
double hitung_diskon(double subtotal, double persentase)
```

```
{
    return subtotal * persentase;
}
```

Fungsi seperti ini disebut fungsi murni secara sederhana: untuk input yang sama, fungsi selalu menghasilkan output yang sama, tanpa mengubah keadaan di luar fungsi. Fungsi murni relatif mudah diuji, dipahami, dan digunakan kembali.

c. Fungsi output

Bertugas menampilkan hasil, tanpa melakukan perhitungan.

```
void tampilkan_hasil(double luas, double keliling)
{
    printf("Luas     : %.2f\n", luas);
    printf("Keliling : %.2f\n", keliling);
}
```

Tidak seluruh fungsi harus murni. Fungsi input dan output tetap memiliki interaksi dengan pengguna, tetapi tanggung jawabnya tetap dijaga agar tidak tercampur dengan perhitungan.

15. Call Flow — Alur Pemanggilan antar Fungsi Modular

Perhatikan:

```
#include <stdio.h>

int kali_dua(int nilai);
int tambah_tiga(int nilai);

int main(void)
{
    int angka = 5;
    int hasil;

    hasil = tambah_tiga(kali_dua(angka));

    printf("Hasil: %d\n", hasil);

    return 0;
}

int kali_dua(int nilai)
{
    return nilai * 2;
}

int tambah_tiga(int nilai)
{
    return nilai + 3;
}
```

Urutan (call flow):

- angka bernilai 5;
- kali_dua(5) dipanggil, mengembalikan 10;
- tambah_tiga(10) dipanggil, mengembalikan 13;
- nilai 13 disimpan dalam hasil.

Untuk mahasiswa pemula, pemanggilan bertingkat dapat ditulis lebih jelas menggunakan variabel sementara:

```
int hasil_kali = kali_dua(angka);
int hasil_akhir = tambah_tiga(hasil_kali);
```

Diagram call flow membantu memvisualisasikan fungsi mana yang memanggil fungsi mana — kemampuan ini akan sangat berguna ketika mempelajari rekursi pada Bagian C, di mana sebuah fungsi memanggil dirinya sendiri berulang kali.

Bagian C — Rekursi

16. Konsep Rekursi

Rekursi adalah teknik dalam pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan bagian dari suatu masalah.

Pada Bagian A dan B, fungsi selalu memanggil fungsi lain (misalnya main() memanggil hitung_total(), yang memanggil hitung_subtotal()). Rekursi memperkenalkan kemungkinan baru: fungsi memanggil dirinya sendiri.

Ide dasarnya: masalah besar diselesaikan dengan memecahnya menjadi masalah yang sama tetapi berukuran lebih kecil, sampai mencapai ukuran yang cukup sederhana untuk dijawab secara langsung.

Contoh sederhana secara matematis:

```
5! = 5 × 4!
4! = 4 × 3!
3! = 3 × 2!
2! = 2 × 1!
1! = 1
```

Perhatikan bahwa setiap baris mendefinisikan faktorial menggunakan faktorial dari bilangan yang lebih kecil, sampai akhirnya mencapai 1! yang nilainya sudah diketahui langsung.

17. Base Case dan Recursive Case

Setiap fungsi rekursif yang benar wajib memiliki dua komponen utama.

a. Base Case (Kondisi Dasar)

Kondisi yang mengakhiri rekursi, agar fungsi berhenti memanggil dirinya sendiri. Tanpa base case, fungsi akan terus memanggil dirinya tanpa batas.

b. Recursive Case (Pemanggilan Rekursif)

Bagian fungsi yang memanggil dirinya sendiri untuk menangani masalah dalam ukuran yang lebih kecil, biasanya mendekati base case setiap kali dipanggil.

Bentuk umum:

```
tipe_kembalian fungsi_rekursif(parameter)
{
```

```
    if (kondisi_dasar)
    {
        return nilai_dasar;   /* base case */
    }

    return fungsi_rekursif(parameter_lebih_kecil);   /* recursive case */
}
```

Setiap pemanggilan rekursif harus membawa fungsi lebih dekat ke base case. Jika tidak, rekursi tidak akan pernah berhenti.

18. Call Stack dan Tracing Rekursi

Setiap kali fungsi dipanggil (termasuk memanggil dirinya sendiri), komputer menyimpan informasi pemanggilan tersebut pada struktur yang disebut call stack.

Contoh fungsi:

```
int hitung_faktorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }

    return n * hitung_faktorial(n - 1);
}
```

Pemanggilan hitung_faktorial(4) menghasilkan tumpukan pemanggilan berikut:

```
hitung_faktorial(4)
  = 4 * hitung_faktorial(3)
        = 3 * hitung_faktorial(2)
              = 2 * hitung_faktorial(1)
                    = 1   <- base case tercapai
```

Kemudian nilai dikembalikan dari tumpukan paling dalam menuju yang paling luar:

```
hitung_faktorial(1) = 1
hitung_faktorial(2) = 2 * 1 = 2
hitung_faktorial(3) = 3 * 2 = 6
hitung_faktorial(4) = 4 * 6 = 24
```

Proses memanggil semakin dalam disebut fase pemanggilan (calling phase), sedangkan proses mengembalikan nilai disebut fase pengembalian (returning phase). Memahami kedua fase ini adalah kunci untuk melakukan tracing rekursi dengan benar.

19. Contoh Rekursi: Countdown

Contoh paling sederhana untuk memahami base case dan recursive case.

```
#include <stdio.h>

void hitung_mundur(int n)
{
    if (n < 0)
    {
        return;   /* base case: berhenti ketika n kurang dari 0 */
    }

    printf("%d\n", n);

    hitung_mundur(n - 1);   /* recursive case */
}
```

Pemanggilan hitung_mundur(3) menghasilkan:

```
3
2
1
0
```

Perhatikan bahwa setelah n bernilai -1, base case tercapai dan fungsi berhenti tanpa mencetak apa pun.

20. Contoh Rekursi: Menjumlahkan Rentang Angka (1 sampai n)

Menjumlahkan dua angka adalah tugas sederhana, tetapi menjumlahkan suatu rentang angka dapat diselesaikan dengan memecahnya menjadi tugas menjumlahkan dua angka secara berulang.

```
int jumlah_1_sampai_n(int n)
{
    if (n == 0)
    {
        return 0;   /* base case */
    }

    return n + jumlah_1_sampai_n(n - 1);   /* recursive case */
}
```

Pemanggilan jumlah_1_sampai_n(4) mengikuti alur berikut:

```
4 + jumlah_1_sampai_n(3)
4 + (3 + jumlah_1_sampai_n(2))
4 + (3 + (2 + jumlah_1_sampai_n(1)))
4 + (3 + (2 + (1 + jumlah_1_sampai_n(0))))
4 + (3 + (2 + (1 + 0)))
= 10
```

Karena fungsi tidak memanggil dirinya lagi saat n bernilai 0, program berhenti di titik tersebut dan mulai mengembalikan hasil akhirnya secara bertahap.

21. Contoh Rekursi: Faktorial

Faktorial dari bilangan n, ditulis n!, adalah hasil perkalian bilangan tersebut dengan semua bilangan bulat positif yang lebih kecil darinya.

```
int hitung_faktorial(int n)
{
    if (n <= 1)
    {
        return 1;   /* base case: 0! dan 1! bernilai 1 */
    }

    return n * hitung_faktorial(n - 1);   /* recursive case */
}
```

Program lengkap:

```
#include <stdio.h>

int hitung_faktorial(int n);

int main(void)
{
    int angka;

    printf("Masukkan bilangan: ");
    scanf("%d", &angka);

    if (angka < 0)
    {
        printf("Faktorial tidak didefinisikan untuk bilangan negatif.\n");
        return 1;
    }

    printf(
        "%d! = %d\n",
        angka,
        hitung_faktorial(angka)
    );

    return 0;
}

int hitung_faktorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }
```

```
    return n * hitung_faktorial(n - 1);
}
```

Jika angka diisi 5, maka hitung_faktorial(5) menghitung 5 × 4 × 3 × 2 × 1 = 120.

Validasi angka < 0 tetap dilakukan di luar fungsi rekursif, karena faktorial secara matematis tidak didefinisikan untuk bilangan negatif — tanpa validasi ini, base case n <= 1 tidak akan pernah tercapai untuk input negatif.

22. Iteratif vs Rekursif

Masalah yang sama sering dapat diselesaikan menggunakan perulangan (iterasi) maupun rekursi.

Versi iteratif dari faktorial:

```
int hitung_faktorial_iteratif(int n)
{
    int hasil = 1;
    int i;

    for (i = 2; i <= n; i++)
    {
        hasil *= i;
    }

    return hasil;
}
```

Perbandingan:

Aspek	Iteratif	Rekursif
Struktur kendali	Perulangan (for/while)	Fungsi memanggil dirinya sendiri
Penggunaan memori	Umumnya tetap	Bertambah seiring kedalaman rekursi (call stack)
Kemudahan penulisan	Kadang lebih panjang	Kadang lebih ringkas untuk masalah hirarkis/berulang
Risiko	Infinite loop jika kondisi salah	Infinite recursion dan stack overflow jika base case salah/tidak ada
Kecepatan	Umumnya lebih cepat	Umumnya lebih lambat karena overhead pemanggilan fungsi

Rekursi membuat kode lebih ringkas terutama untuk masalah yang secara alami berstruktur hirarkis atau berulang (misalnya traversal pohon atau pembagian wilayah pada binary search). Namun, rekursi membutuhkan lebih banyak memori karena setiap pemanggilan disimpan pada call stack, dan jika tidak berhenti dapat menyebabkan stack overflow.

23. Infinite Recursion

Infinite recursion terjadi ketika fungsi rekursif tidak pernah mencapai base case, sehingga terus memanggil dirinya sendiri tanpa henti.

Contoh yang salah — base case terhapus:

```
int hitung_faktorial_salah(int n)
{
    return n * hitung_faktorial_salah(n - 1);
}
```

Fungsi tersebut tidak pernah berhenti, karena tidak ada kondisi if yang mengembalikan nilai secara langsung. Setiap pemanggilan membuat satu bingkai (frame) baru pada call stack. Karena call stack memiliki ukuran terbatas, program akan berhenti secara tidak normal akibat stack overflow.

Contoh lain — base case ada, tetapi tidak pernah tercapai:

```
int hitung_mundur_salah(int n)
{
    if (n == 0)
    {
        return 0;
    }

    return hitung_mundur_salah(n + 1);   /* menjauh dari base case! */
}
```

Pada contoh ini, base case n == 0 memang ada, tetapi recursive case n + 1 membuat n semakin menjauh dari base case apabila dipanggil dengan nilai awal yang tidak akan pernah mencapai 0 secara pasti (misalnya dengan nilai awal negatif). Pastikan recursive case selalu bergerak mendekati base case.

24. Fibonacci sebagai Contoh Inefisiensi Rekursi

Barisan Fibonacci adalah barisan bilangan di mana setiap suku merupakan penjumlahan dua suku sebelumnya.

```
int fibonacci(int n)
{
    if ((n == 0) || (n == 1))
    {
        return n;   /* base case */
    }

    return fibonacci(n - 1) + fibonacci(n - 2);   /* recursive case */
}
```

Berbeda dengan faktorial atau countdown, fungsi ini memanggil dirinya sendiri dua kali pada setiap pemanggilan (bukan satu kali). Hal ini menyebabkan jumlah pemanggilan bertambah sangat cepat seiring bertambahnya n.

Ilustrasi pemanggilan fibonacci(4):

```
                  fib(4)
                /       \
            fib(3)         fib(2)
           /      \       /      \
       fib(2)   fib(1)  fib(1)  fib(0)
       /    \
   fib(1)  fib(0)
```

Perhatikan bahwa fib(2) dan fib(1) dihitung ulang beberapa kali secara terpisah — inilah sumber inefisiensi rekursi naif pada Fibonacci. Untuk n yang besar, jumlah pemanggilan bertambah secara eksponensial.

Inilah salah satu alasan mengapa rekursi sering kali lebih lambat daripada iterasi dalam beberapa kasus: rekursi dapat menghitung ulang submasalah yang sama berkali-kali, sedangkan versi iteratif biasanya hanya menghitung setiap nilai sekali.

Teknik untuk mengatasi inefisiensi ini (misalnya memoization) berada di luar cakupan modul ini dan dapat dipelajari lebih lanjut pada mata kuliah Struktur Data atau Analisis Algoritma.

IV. Praktikum Terbimbing

1. Praktikum Terbimbing 1 — Fungsi Dasar

Buat berkas: latihan_01.c. Program:

```c
#include <stdio.h>

void tampilkan_judul(void);
double hitung_luas(double panjang, double lebar);
double hitung_keliling(double panjang, double lebar);
void tampilkan_hasil(
    double panjang,
    double lebar,
    double luas,
    double keliling
);

int main(void)
{
    double panjang;
    double lebar;
    double luas;
    double keliling;

    tampilkan_judul();

    printf("Masukkan panjang: ");
    scanf("%lf", &panjang);

    printf("Masukkan lebar: ");
    scanf("%lf", &lebar);
```

```
    if ((panjang <= 0.0) || (lebar <= 0.0))
    {
        printf("Panjang dan lebar harus lebih besar dari nol.\n");
        return 1;
    }

    luas = hitung_luas(panjang, lebar);
    keliling = hitung_keliling(panjang, lebar);

    tampilkan_hasil(panjang, lebar, luas, keliling);

    return 0;
}

void tampilkan_judul(void)
{
    printf("========================================\n");
    printf("     PRAKTIKUM FUNGSI DAN MODULAR       \n");
    printf("========================================\n");
}

double hitung_luas(double panjang, double lebar)
{
    return panjang * lebar;
}

double hitung_keliling(double panjang, double lebar)
{
    return 2.0 * (panjang + lebar);
}

void tampilkan_hasil(
    double panjang,
    double lebar,
    double luas,
    double keliling
)
{
    printf("\nHASIL PERHITUNGAN\n");
    printf("Panjang  : %.2f\n", panjang);
    printf("Lebar    : %.2f\n", lebar);
    printf("Luas     : %.2f\n", luas);
    printf("Keliling : %.2f\n", keliling);
}
```

2. Praktikum Terbimbing 2 — Modularisasi

Buat berkas: latihan_02.c. Program awal berikut belum modular:

```
#include <stdio.h>

int main(void)
{
    int total_detik;
    int jam;
    int menit;
    int detik;
    int sisa;

    printf("Masukkan total detik: ");
    scanf("%d", &total_detik);

    if (total_detik < 0)
    {
        printf("Total detik tidak boleh negatif.\n");
        return 1;
    }

    jam = total_detik / 3600;
    sisa = total_detik % 3600;
    menit = sisa / 60;
    detik = sisa % 60;

    printf("%d detik setara dengan:\n", total_detik);
    printf("%d jam\n", jam);
    printf("%d menit\n", menit);
    printf("%d detik\n", detik);

    return 0;
}
```

Ubah menjadi program modular menggunakan fungsi:

```
int hitung_jam(int total_detik);
int hitung_sisa_setelah_jam(int total_detik);
int hitung_menit(int sisa_detik);
int hitung_detik(int sisa_detik);
void tampilkan_waktu(
    int total_detik,
    int jam,
    int menit,
    int detik
);
```

3. Praktikum Terbimbing 3 — Rekursi

Buat berkas: latihan_03.c. Program:

```
#include <stdio.h>

void hitung_mundur(int n);
int hitung_faktorial(int n);
int jumlah_1_sampai_n(int n);

int main(void)
{
    int angka;

    printf("Masukkan bilangan: ");
    scanf("%d", &angka);

    if (angka < 0)
    {
        printf("Bilangan tidak boleh negatif.\n");
        return 1;
    }

    printf("\nHitung mundur:\n");
    hitung_mundur(angka);

    printf(
        "\n%d! = %d\n",
        angka,
        hitung_faktorial(angka)
    );

    printf(
        "1 + 2 + ... + %d = %d\n",
        angka,
        jumlah_1_sampai_n(angka)
    );

    return 0;
}

void hitung_mundur(int n)
{
    if (n < 0)
    {
        return;
    }

    printf("%d\n", n);
```

```
    hitung_mundur(n - 1);
}

int hitung_faktorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }

    return n * hitung_faktorial(n - 1);
}

int jumlah_1_sampai_n(int n)
{
    if (n == 0)
    {
        return 0;
    }

    return n + jumlah_1_sampai_n(n - 1);
}
```

V. Eksperimen dan Tracing

1. Tracing Pemanggilan Fungsi

Buat berkas: tracing.c. Jangan langsung menjalankan program.

```
#include <stdio.h>

int tambah(int a, int b);
int kali(int a, int b);
int proses(int nilai);

int main(void)
{
    int x = 4;
    int y = 3;
    int hasil;

    hasil = proses(tambah(x, y));

    printf("Hasil akhir: %d\n", hasil);

    return 0;
```

```
}

int tambah(int a, int b)
{
    return a + b;
}

int kali(int a, int b)
{
    return a * b;
}

int proses(int nilai)
{
    int hasil_sementara = kali(nilai, 2);

    return hasil_sementara + 1;
}
```

Lengkapi tabel tracing:

Langkah	Fungsi Aktif	Pernyataan	Nilai Parameter/Variabel	Nilai Kembalian
1	main()	x = 4	x = 4	–
2	main()	y = 3	y = 3	–
3	tambah()	tambah(x, y)	a = 4, b = 3	
4	proses()	proses(...)	nilai = ...	
5	kali()	kali(nilai, 2)	a = ..., b = 2	
6	proses()	hasil_sementara + 1	hasil_sementara = ...	
7	main()	hasil = ...	hasil = ...	–

Tracing Rekursi: hitung_faktorial(3)

Lengkapi tabel tracing fase pemanggilan (calling phase) dan fase pengembalian (returning phase):

Fase	Pemanggilan	n	Base Case Tercapai?	Nilai Dikembalikan
Panggil	hitung_faktorial(3)	3	Tidak	menunggu ...
Panggil	hitung_faktorial(2)	2	Tidak	menunggu ...
Panggil	hitung_faktorial(1)	1	Ya	
Kembali	hitung_faktorial(1)	1	–	
Kembali	hitung_faktorial(2)	2	–	
Kembali	hitung_faktorial(3)	3	–	

Setelah tracing:

- prediksi output kedua program;
- jalankan program;
- bandingkan hasil;
- jelaskan kapan setiap variabel lokal dibuat dan selesai digunakan;
- jelaskan perbedaan antara fase pemanggilan dan fase pengembalian pada tracing rekursi.

VI. Debugging Challenge

1. Debugging Challenge 1 — Prototype dan Nilai Kembalian Tidak Sesuai

Program berikut dimaksudkan untuk menghitung luas dan keliling lingkaran.

```
#include <stdio.h>

double hitung_luas(int jari_jari);
void hitung_keliling(double jari_jari);

int main(void)
{
    double jari_jari;
    double luas;
    double keliling;

    printf("Masukkan jari-jari: ");
    scanf("%lf", &jari_jari);

    luas = hitung_luas(jari_jari);
    keliling = hitung_keliling(jari_jari);

    printf("Luas      : %.2f\n", luas);
    printf("Keliling  : %.2f\n", keliling);

    return 0;
}

int hitung_luas(double jari_jari)
{
    const double PHI = 3.141592653589793;

    return PHI * jari_jari * jari_jari;
}

double hitung_keliling(double jari_jari)
{
    const double PHI = 3.141592653589793;

    PHI * 2.0 * jari_jari;
}
```

Kesalahan yang perlu ditemukan

- tipe parameter prototype tidak sesuai definisi;
- tipe kembalian prototype tidak sesuai definisi;
- fungsi hitung_keliling() dideklarasikan void, tetapi digunakan sebagai nilai;
- definisi hitung_luas() menggunakan tipe kembalian berbeda;
- hitung_keliling() tidak menggunakan return;

- hasil ekspresi pada hitung_keliling() tidak dikembalikan;
- validasi jari-jari belum dilakukan.

Tugas

- kompilasi program dengan warning aktif;
- catat seluruh error dan warning;
- perbaiki prototype;
- perbaiki tipe kembalian;
- tambahkan pernyataan return;
- tambahkan validasi jari-jari;
- uji menggunakan jari-jari 1, 7, dan 10.

2. Debugging Challenge 2 — Infinite Recursion

Program berikut dimaksudkan untuk menghitung jumlah digit suatu bilangan positif menggunakan rekursi.

```
#include <stdio.h>

int jumlah_digit(int n);

int main(void)
{
    int angka;

    printf("Masukkan bilangan: ");
    scanf("%d", &angka);

    printf(
        "Jumlah digit: %d\n",
        jumlah_digit(angka)
    );

    return 0;
}

int jumlah_digit(int n)
{
    return (n % 10) + jumlah_digit(n / 10);
}
```

Coba jalankan program ini dengan input, misalnya, 123. Amati apa yang terjadi (program mungkin berjalan sangat lama, menampilkan hasil yang aneh, atau berhenti tiba-tiba akibat stack overflow).

Kesalahan yang perlu ditemukan

- fungsi jumlah_digit() tidak memiliki base case sama sekali;
- setiap pemanggilan jumlah_digit(n / 10) tetap memanggil dirinya sendiri meskipun n sudah menjadi satu digit atau bahkan 0;
- ketika n mencapai 0, n / 10 tetap menghasilkan 0, sehingga fungsi memanggil jumlah_digit(0) tanpa henti;

- tidak ada input negatif yang divalidasi.

Perbaikan Inti

```
int jumlah_digit(int n)
{
    if (n < 10)
    {
        return n;   /* base case: satu digit tersisa */
    }

    return (n % 10) + jumlah_digit(n / 10);
}
```

Tugas

- identifikasi mengapa fungsi awal tidak memiliki base case;
- jelaskan apa yang terjadi pada call stack ketika base case tidak pernah tercapai;
- perbaiki fungsi dengan menambahkan base case yang tepat;
- uji dengan input 5 (satu digit), 123 (tiga digit), dan 0;
- jelaskan mengapa program awal berpotensi menyebabkan stack overflow, bukan sekadar jawaban yang salah.

VII. Latihan Mandiri

1. Level 1 — Fungsi Kuadrat dan Kubik

Buat fungsi:

```
int hitung_kuadrat(int bilangan);
int hitung_kubik(int bilangan);
```

Program menerima satu bilangan dan menampilkan nilai awal, kuadrat, dan kubik.

Contoh:

```
Masukkan bilangan: 4
Kuadrat: 16
Kubik  : 64
```

2. Level 2 — Fungsi Bilangan Genap

Buat fungsi:

```
int apakah_genap(int bilangan);
```

Fungsi mengembalikan 1 apabila bilangan genap, 0 apabila bilangan ganjil.

Gunakan fungsi tersebut untuk memeriksa sepuluh bilangan yang dimasukkan pengguna. Program menampilkan jumlah bilangan genap dan ganjil.

3. Level 3 — Kalkulator Modular

Buat fungsi:

```
double tambah(double a, double b);
double kurang(double a, double b);
```

```
double kali(double a, double b);
double bagi(double a, double b);
void tampilkan_menu(void);
```

Gunakan switch untuk memilih operasi. Program wajib memeriksa pembagian dengan nol.

4. Level 4 — Rekursi: Pangkat Sederhana

Buat fungsi rekursif:

```
int hitung_pangkat(int basis, int pangkat);
```

Fungsi menghitung basis pangkat menggunakan rekursi, tanpa menggunakan pow() atau perulangan.

Ketentuan:

- base case: pangkat == 0 mengembalikan 1;
- recursive case: basis dikalikan dengan hasil hitung_pangkat(basis, pangkat - 1);
- pangkat yang diterima selalu bernilai 0 atau lebih (validasi di luar fungsi rekursif).

Contoh:

```
Masukkan basis: 3
Masukkan pangkat: 4
3^4 = 81
```

5. Challenge — Fungsi Bilangan Prima

Buat fungsi:

```
int apakah_prima(int bilangan);
```

Fungsi mengembalikan 1 apabila bilangan prima, 0 apabila bukan bilangan prima.

Ketentuan:

- bilangan kurang dari 2 bukan bilangan prima;
- pemeriksaan pembagi dilakukan menggunakan perulangan (bukan rekursi);
- cukup memeriksa pembagi sampai kondisi: pembagi * pembagi <= bilangan.

Gunakan fungsi tersebut untuk menampilkan seluruh bilangan prima dari 1 sampai batas yang dimasukkan pengguna.

6. Jumlah Digit Rekursif

Keluaran program pada soal ini akan diperiksa secara ketat: harus sama persis karakter demi karakter dengan Contoh Output.

Sebuah aplikasi validasi ingin menghitung jumlah digit (penjumlahan nilai setiap angka penyusun) dari sebuah bilangan. Buat program C dengan fungsi rekursif int jumlah_digit(int n) yang menghitung jumlah digit tersebut, sesuai pendekatan pada Debugging Challenge 2 (materi VI.2).

Format Input

Satu baris bilangan bulat non-negatif: n.

Batasan

- $0 \leq n \leq 999999999$.

Ketentuan

Base case: apabila n kurang dari 10 (satu digit tersisa), kembalikan n secara langsung. Recursive case: kembalikan (n % 10) + jumlah_digit(n / 10). Jangan menggunakan perulangan atau array — gunakan rekursi murni.

Contoh 1

Input:

```
12345
```

Output:

```
=======================================
         JUMLAH DIGIT REKURSIF
=======================================
Bilangan     : 12345
---------------------------------------
Jumlah digit : 15
=======================================
```

Contoh 2

Input:

```
0
```

Output:

```
=======================================
         JUMLAH DIGIT REKURSIF
=======================================
Bilangan     : 0
---------------------------------------
Jumlah digit : 0
=======================================
```

Contoh 3

Input:

```
9
```

Output:

```
=======================================
         JUMLAH DIGIT REKURSIF
=======================================
Bilangan     : 9
---------------------------------------
Jumlah digit : 9
=======================================
```

Ketentuan Tambahan

- Contoh 2 dan Contoh 3 menunjukkan kasus bilangan satu digit (termasuk 0) — base case n < 10 harus langsung tercapai tanpa pemanggilan rekursif tambahan;

- kedua baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output.

Simpan sebagai latihan_bonus_07.c.

VIII. Tugas Praktikum

1. Tugas Praktikum Modul 7 — Program Rekap Belanja Modular

Buat program untuk menerima beberapa data barang dan menghitung ringkasan transaksi menggunakan fungsi-fungsi yang terpisah.

Tugas ini tetap berfokus pada modularisasi (Bagian A dan B). Sebagai penerapan Bagian C, program wajib menggunakan minimal satu fungsi rekursif untuk menjumlahkan subtotal seluruh barang, menggantikan akumulasi menggunakan +=.

1. Input

Program menerima: jumlah jenis barang, harga satuan setiap barang, jumlah unit setiap barang, status member.

Batasan:

- jumlah jenis barang: 1–20;
- harga satuan: lebih besar dari 0;
- jumlah unit: 1–1000;
- status member: 0 bukan member, 1 member.

2. Ketentuan Perhitungan

Subtotal setiap barang:

```
subtotal barang = harga satuan × jumlah unit
```

Total belanja (dihitung secara rekursif, lihat bagian 4):

```
total belanja =
jumlah seluruh subtotal barang
```

Ketentuan diskon:

Kondisi	Diskon
Bukan member	0%
Member dan total < Rp100.000	5%
Member dan total Rp100.000–kurang dari Rp500.000	10%
Member dan total ≥ Rp500.000	15%

Biaya layanan: 5% dari nilai setelah diskon.

Total pembayaran:

```
nilai setelah diskon =
total belanja – diskon

biaya layanan =
nilai setelah diskon × 5%

total pembayaran =
nilai setelah diskon + biaya layanan
```

3. Penyimpanan Subtotal dan Penjumlahan Rekursif

Karena total belanja sekarang dihitung menggunakan rekursi (bukan akumulasi langsung di dalam perulangan input), setiap subtotal barang harus disimpan terlebih dahulu ke dalam array.

```
#define MAX_BARANG 20

double subtotal_barang[MAX_BARANG];
```

Setelah seluruh subtotal barang tersimpan dalam array, gunakan fungsi rekursif berikut untuk menjumlahkannya:

```
double hitung_total_rekursif(
    const double subtotal[],
    int jumlah_data
)
{
    if (jumlah_data <= 0)
    {
        return 0.0;   /* base case */
    }

    return
        subtotal[jumlah_data - 1] +
        hitung_total_rekursif(
            subtotal,
            jumlah_data - 1
        );   /* recursive case */
}
```

Fungsi ini menelusuri array dari elemen terakhir menuju elemen pertama, menjumlahkannya secara rekursif hingga jumlah_data mencapai 0 (base case).

Perhatikan bahwa array dan penelusurannya akan dibahas lebih mendalam pada Modul 8. Pada modul ini, mahasiswa cukup memahami bahwa array dapat ditelusuri secara rekursif seperti halnya secara iteratif.

4. Fungsi yang Wajib Dibuat

Program wajib memiliki minimal fungsi berikut:

```
int baca_bilangan_dalam_rentang(
    int nilai_minimum,
    int nilai_maksimum
);

double baca_bilangan_positif(void);

double hitung_subtotal_barang(
    double harga_satuan,
    int jumlah_unit
);

double hitung_total_rekursif(
```

```
    const double subtotal[],
    int jumlah_data
);

double hitung_persentase_diskon(
    double total_belanja,
    int status_member
);

double hitung_nilai_diskon(
    double total_belanja,
    double persentase_diskon
);

double hitung_biaya_layanan(
    double nilai_setelah_diskon
);

double hitung_total_pembayaran(
    double nilai_setelah_diskon,
    double biaya_layanan
);

void tampilkan_ringkasan(
    double total_belanja,
    double persentase_diskon,
    double nilai_diskon,
    double biaya_layanan,
    double total_pembayaran
);
```

Mahasiswa boleh menambahkan fungsi lain apabila diperlukan.

5. Contoh Interaksi

```
================================================
             PROGRAM REKAP BELANJA
================================================
Masukkan jumlah jenis barang 1-20: 3

BARANG KE-1
Harga satuan : 20000
Jumlah unit  : 2
Subtotal     : Rp40000.00

BARANG KE-2
Harga satuan : 50000
Jumlah unit  : 3
```

```
Subtotal     : Rp150000.00

BARANG KE-3
Harga satuan : 10000
Jumlah unit  : 1
Subtotal     : Rp10000.00

Status member:
0. Bukan member
1. Member
Masukkan status: 1

RINGKASAN TRANSAKSI
------------------------------------------------
Total belanja       : Rp200000.00
Persentase diskon   : 10.00%
Nilai diskon        : Rp20000.00
Nilai setelah diskon: Rp180000.00
Biaya layanan       : Rp9000.00
Total pembayaran    : Rp189000.00
================================================
```

Nilai Total belanja Rp200000.00 diperoleh dari hitung_total_rekursif() menjumlahkan array subtotal [40000.00, 150000.00, 10000.00], bukan dari akumulasi += di dalam perulangan input.

6. Tahapan Algoritma

1. menampilkan judul;
2. menerima jumlah jenis barang;
3. memvalidasi jumlah jenis barang;
4. melakukan perulangan untuk setiap barang;
5. menerima harga satuan;
6. memvalidasi harga;
7. menerima jumlah unit;
8. memvalidasi jumlah unit;
9. menghitung subtotal melalui fungsi;
10. menyimpan subtotal ke dalam array;
11. menghitung total belanja menggunakan hitung_total_rekursif();
12. menerima status member;
13. menentukan persentase diskon melalui fungsi;
14. menghitung nilai diskon;
15. menghitung nilai setelah diskon;
16. menghitung biaya layanan;
17. menghitung total pembayaran;
18. menampilkan ringkasan melalui fungsi.

7. Kerangka Program

```
#include <stdio.h>

#define MAX_BARANG 20

int baca_bilangan_dalam_rentang(
    int nilai_minimum,
    int nilai_maksimum
);

double baca_bilangan_positif(void);

double hitung_subtotal_barang(
    double harga_satuan,
    int jumlah_unit
);

double hitung_total_rekursif(
    const double subtotal[],
    int jumlah_data
);

/* Lengkapi prototype fungsi lainnya. */

int main(void)
{
    int jumlah_jenis_barang;
    int nomor_barang;
    int jumlah_unit;
    int status_member;

    double harga_satuan;
    double subtotal_barang[MAX_BARANG];
    double total_belanja;
    double persentase_diskon;
    double nilai_diskon;
    double nilai_setelah_diskon;
    double biaya_layanan;
    double total_pembayaran;

    printf("Masukkan jumlah jenis barang 1-20: ");
    jumlah_jenis_barang =
        baca_bilangan_dalam_rentang(1, MAX_BARANG);

    for (nomor_barang = 1;
         nomor_barang <= jumlah_jenis_barang;
         nomor_barang++)
    {
```

```
        printf("\nBARANG KE-%d\n", nomor_barang);

        printf("Harga satuan: ");
        harga_satuan = baca_bilangan_positif();

        printf("Jumlah unit 1-1000: ");
        jumlah_unit =
            baca_bilangan_dalam_rentang(1, 1000);

        subtotal_barang[nomor_barang - 1] =
            hitung_subtotal_barang(
                harga_satuan,
                jumlah_unit
            );

        printf(
            "Subtotal: Rp%.2f\n",
            subtotal_barang[nomor_barang - 1]
        );
    }

    total_belanja = hitung_total_rekursif(
        subtotal_barang,
        jumlah_jenis_barang
    );

    printf("\nStatus member 0/1: ");
    status_member =
        baca_bilangan_dalam_rentang(0, 1);

    /* Tentukan persentase diskon. */

    /* Hitung nilai diskon. */

    /* Hitung nilai setelah diskon. */

    /* Hitung biaya layanan. */

    /* Hitung total pembayaran. */

    /* Tampilkan ringkasan. */

    return 0;
}

/* Lengkapi definisi seluruh fungsi. */
```

8. Analisis Dekomposisi

Mahasiswa wajib melengkapi tabel berikut.

Fungsi	Input/Parameter	Proses	Nilai Kembalian/Output
baca_bilangan_dalam_rentang()			
baca_bilangan_positif()			
hitung_subtotal_barang()			
hitung_total_rekursif()			
hitung_persentase_diskon()			
hitung_nilai_diskon()			
hitung_biaya_layanan()			
hitung_total_pembayaran()			
tampilkan_ringkasan()			

9. Diagram Pemanggilan Fungsi

Mahasiswa membuat diagram seperti berikut dan melengkapinya:

```
main()
├── baca_bilangan_dalam_rentang()
├── baca_bilangan_positif()
├── hitung_subtotal_barang()
├── hitung_total_rekursif()
│     └── hitung_total_rekursif()  (memanggil dirinya sendiri)
├── hitung_persentase_diskon()
├── hitung_nilai_diskon()
├── hitung_biaya_layanan()
├── hitung_total_pembayaran()
└── tampilkan_ringkasan()
```

Jelaskan: fungsi mana yang dipanggil berulang di dalam perulangan, fungsi mana yang bersifat rekursif, fungsi mana yang hanya dipanggil sekali, dan fungsi mana yang bertipe void.

10. Ketentuan Source Code

Program wajib:

- menggunakan #include <stdio.h>;
- menggunakan int main(void);
- memiliki function prototype;
- memiliki minimal sembilan fungsi;
- menggunakan fungsi bertipe void;
- menggunakan fungsi dengan nilai kembalian;
- menggunakan fungsi dengan parameter;
- menggunakan validasi input dalam fungsi;
- menggunakan pass by value;
- tidak bergantung pada variabel global;

- menggunakan nama fungsi yang deskriptif;
- menggunakan satu tanggung jawab utama untuk setiap fungsi;
- menggunakan perulangan untuk memproses barang;
- menyimpan subtotal setiap barang ke dalam array;
- menggunakan fungsi rekursif (hitung_total_rekursif()) untuk menjumlahkan subtotal, bukan akumulasi += secara langsung;
- memastikan fungsi rekursif memiliki base case yang jelas;
- menggunakan konstanta untuk biaya layanan dan kapasitas array;
- dikompilasi dengan warning aktif;
- tidak menghasilkan error atau warning penting;
- diuji menggunakan minimal sepuluh test case;
- disertai tracing pemanggilan fungsi, termasuk tracing pemanggilan rekursif;
- dapat dijelaskan oleh mahasiswa.

11. Header Komentar

```
/*
Nama        :
NIM         :
Kelas       :
Modul       : 7
Deskripsi   : Program rekap belanja yang dikembangkan
              menggunakan fungsi, pemrograman modular,
              dan penjumlahan total secara rekursif.
*/
```

12. Berkas yang Dikumpulkan

- source code: tugas_modul_07_nim.c
- analisis input–process–output;
- tabel dekomposisi fungsi;
- function prototype;
- diagram pemanggilan fungsi;
- pseudocode atau flowchart;
- trace table untuk satu data uji, termasuk tracing hitung_total_rekursif();
- tabel pengujian minimal sepuluh kasus;
- tangkapan layar kompilasi;
- tangkapan layar eksekusi;
- catatan debugging;
- refleksi;
- pernyataan penggunaan AI.

2. Contoh Test Case Tugas

No.	Kondisi	Expected Result
1	Jumlah barang 0	Input diminta ulang
2	Jumlah barang 21	Input diminta ulang
3	Harga 0	Input diminta ulang
4	Harga negatif	Input diminta ulang
5	Jumlah unit 0	Input diminta ulang
6	Bukan member, total Rp50.000	Diskon 0%
7	Member, total Rp50.000	Diskon 5%
8	Member, total Rp100.000	Diskon 10%
9	Member, total Rp499.999	Diskon 10%
10	Member, total Rp500.000	Diskon 15%
11	Status member 2	Input diminta ulang
12	Satu barang: Rp20.000 × 3	Subtotal Rp60.000, total belanja Rp60.000 (hasil hitung_total_rekursif() dengan satu elemen)
13	20 barang (kapasitas maksimum)	hitung_total_rekursif() tetap benar, tidak terjadi akses di luar array

Mahasiswa harus menghitung expected output secara manual untuk setiap test case.

3. Pangkat Rekursif dengan Jumlah Langkah

*Buat program C dengan fungsi rekursif int pangkat_rekursif(int basis, int pangkat, int *jumlah_langkah) yang menghitung basis pangkat pangkat, sekaligus menghitung jumlah pemanggilan rekursif (termasuk pemanggilan yang mencapai base case) melalui parameter output, sesuai pendekatan pada materi III.29 dan pola parameter output yang telah dipelajari pada Modul 7 sebelumnya (fungsi output melalui parameter). Keluaran program dinilai secara ketat: harus sama persis karakter demi karakter, termasuk spasi.*

Format Input

Baris pertama: dua bilangan bulat basis dan pangkat, dipisah spasi.

Batasan

- $-100 \leq basis \leq 100$;
- $0 \leq pangkat \leq 20$.

Ketentuan

Base case: apabila pangkat == 0, kembalikan 1 dan hitung pemanggilan ini sebagai satu langkah. Recursive case: kembalikan basis dikalikan pangkat_rekursif(basis, pangkat - 1, jumlah_langkah), dan tambahkan jumlah_langkah pada setiap pemanggilan (termasuk pemanggilan yang mencapai base case).

Contoh 1

Input:

```
2 5
```

Output:

```
=======================================
    PANGKAT REKURSIF DENGAN LANGKAH
=======================================
Basis          : 2
Pangkat        : 5
---------------------------------------
Hasil          : 32
Jumlah langkah : 6
=======================================
```

Contoh 2

Input:

```
3 0
```

Output:

```
=======================================
    PANGKAT REKURSIF DENGAN LANGKAH
=======================================
Basis          : 3
Pangkat        : 0
---------------------------------------
Hasil          : 1
Jumlah langkah : 1
=======================================
```

Contoh 3

Input:

```
-2 3
```

Output:

```
=======================================
    PANGKAT REKURSIF DENGAN LANGKAH
=======================================
Basis          : -2
Pangkat        : 3
---------------------------------------
Hasil          : -8
Jumlah langkah : 4
=======================================
```

Ketentuan Tambahan

- Contoh 2 menunjukkan pangkat = 0 — base case tercapai pada pemanggilan pertama, sehingga jumlah_langkah harus 1, bukan 0;
- Contoh 3 menunjukkan basis negatif — pastikan tipe data dan operasi perkalian menangani tanda negatif dengan benar (hasil (-2)^3 = -8);
- kedua baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output.

Simpan sebagai tugas_modul_07_soal2_nim.c dan sertakan sebagai lampiran tambahan bersama tugas utama.

X. Kesalahan Umum dan Standar Penulisan

1. Kesalahan yang Sering Terjadi

a. Prototype tidak sesuai definisi

Salah:

```
double hitung_luas(int sisi);
```

Definisi:

```
double hitung_luas(double sisi)
{
    return sisi * sisi;
}
```

Tipe parameter harus konsisten.

b. Lupa titik koma pada prototype

Salah:

```
double hitung_luas(double sisi)
```

Benar:

```
double hitung_luas(double sisi);
```

c. Menambahkan titik koma setelah definisi

Tidak diperlukan:

```
double hitung_luas(double sisi)
{
    return sisi * sisi;
};
```

d. Fungsi non-void tidak mengembalikan nilai

Salah:

```
double hitung_luas(double sisi)
{
    sisi * sisi;
}
```

Benar:

```
double hitung_luas(double sisi)
{
    return sisi * sisi;
}
```

e. Menggunakan hasil fungsi void

Salah:

```
double hasil = tampilkan_menu();
```

Fungsi void tidak menghasilkan nilai.

f. Jumlah atau urutan argumen tidak sesuai

```
double hitung_luas(double panjang, double lebar);
```

Salah:

```
luas = hitung_luas(panjang);
```

Pemanggilan berikut juga berbeda maknanya:

```
hitung_pembagian(10.0, 2.0);
```

dan:

```
hitung_pembagian(2.0, 10.0);
```

g. Mengharapkan parameter mengubah argumen

```
void ubah(int nilai)
{
    nilai = 100;
}
```

Nilai pada pemanggil tetap tidak berubah karena parameter dikirim berdasarkan nilai (pass by value).

h. Menggunakan variabel lokal di luar fungsi

Variabel lokal hanya tersedia di dalam fungsi tempat variabel tersebut dideklarasikan.

i. Fungsi melakukan terlalu banyak pekerjaan

Kurang baik:

```
void input_hitung_tampilkan_semua(void);
```

Lebih baik dipisahkan menjadi beberapa fungsi dengan satu tanggung jawab masing-masing.

j. Fungsi rekursif tanpa base case

Salah:

```
int hitung_faktorial_salah(int n)
{
    return n * hitung_faktorial_salah(n - 1);
}
```

Tanpa base case, fungsi tidak pernah berhenti memanggil dirinya sendiri hingga terjadi stack overflow.

k. Recursive case tidak bergerak menuju base case

Salah:

```
int mundur_salah(int n)
{
    if (n == 0) { return 0; }
    return mundur_salah(n + 1);
}
```

Base case ada, tetapi recursive case menjauh darinya sehingga tetap menyebabkan infinite recursion untuk input tertentu.

l. Base case salah letak atau salah kondisi

Base case yang diperiksa setelah recursive case, atau menggunakan kondisi yang salah (misalnya == alih-alih <=), dapat menyebabkan base case terlewat untuk sebagian nilai input.

m. Kode setelah return

```
int contoh(void)
{
    return 1;

    printf("Tidak dijalankan.\n");
}
```

Kode setelah return tidak dapat dijangkau.

n. Nama fungsi tidak informatif

Kurang baik:

```
double proses(double a, double b);
```

Lebih baik:

```
double hitung_total_bayar(
    double subtotal,
    double diskon
);
```

2. Standar Penulisan Fungsi

Gunakan kata kerja pada nama fungsi

```
hitung_luas
baca_nilai
tentukan_status
tampilkan_hasil
validasi_input
```

Gunakan satu tanggung jawab utama

```
double hitung_diskon(
    double subtotal,
    double persentase
```

```
)
{
    return subtotal * persentase;
}
```

Batasi panjang fungsi

Apabila satu fungsi terlalu panjang, periksa apakah dapat dipecah menjadi beberapa subfungsi.

Hindari efek samping yang tidak perlu

Fungsi perhitungan sebaiknya tidak sekaligus mengubah data global atau menampilkan banyak keluaran.

Jaga konsistensi tipe

Prototype, definisi, parameter, argumen, dan nilai kembalian harus sesuai.

Pastikan setiap fungsi rekursif memiliki base case yang jelas dan tercapai

Tuliskan base case sebagai kondisi pertama yang diperiksa di dalam fungsi, dan pastikan setiap recursive case membawa parameter lebih dekat ke base case.

```
int fungsi_rekursif(int n)
{
    if (n <= 0)   /* base case diperiksa lebih dahulu */
    {
        return 0;
    }

    return n + fungsi_rekursif(n - 1);   /* n - 1 mendekati base case */
}
```

Berikan komentar pada tujuan fungsi

```
/*
Menghitung nilai diskon berdasarkan subtotal
dan persentase dalam bentuk desimal.
*/
double hitung_diskon(
    double subtotal,
    double persentase
)
{
    return subtotal * persentase;
}
```

Komentar tidak perlu mengulang setiap baris source code.

XII. Pernyataan Penggunaan AI

Tidak Menggunakan AI:

```
Saya menyatakan bahwa tugas ini dikerjakan tanpa menggunakan bantuan
AI generatif.
```

Menggunakan AI:

```
Saya menggunakan [nama perangkat AI] untuk membantu menjelaskan
[prototype, parameter, nilai kembalian, rekursi, base case, error,
atau bagian lain]. Hasil tersebut saya periksa melalui tracing,
kompilasi, pengujian setiap fungsi, dan perbandingan dengan
algoritma yang disusun.
```

Daftar Pustaka

Deitel, P., & Deitel, H. (2016). C: How to program (8th ed.). Pearson Education.

Free Software Foundation. (n.d.). GCC, the GNU Compiler Collection: Online documentation. GNU Project. Diakses dari https://gcc.gnu.org/onlinedocs/

International Organization for Standardization. (2018). ISO/IEC 9899:2018 — Information technology — Programming languages — C. ISO.

Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.

McConnell, S. (2004). Code complete: A practical handbook of software construction (2nd ed.). Microsoft Press.

Microsoft. (n.d.). C/C++ for Visual Studio Code. Visual Studio Code Documentation. Diakses dari https://code.visualstudio.com/docs/languages/cpp

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. Communications of the ACM, 15(12), 1053–1058.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.